



Handout

Dateiformate XML und JSON

Inhalt

1	Einführung	3
2	XML (Extensible Markup Language)	3
2.1	Definition und Zweck	3
2.2	Struktur und Syntax.....	3
2.3	Datentypen	4
2.4	Vorteile von XML.....	4
2.5	Nachteile von XML	6
2.6	Anwendungsbereiche von XML	6
3	JSON (JavaScript Object Notation)	7
3.1	Definition und Zweck	7
3.2	Struktur und Syntax.....	7
3.3	Datentypen	8
3.4	Vorteile von JSON	8
3.5	Nachteile von JSON	8
3.6	Anwendungsbereiche von JSON	9
4	Vergleichstabelle: XML vs. JSON	10
5	Komplexere Beispiele	11
5.1	Beispiel: XML – Stadtbibliothek.....	11
5.2	Beispiel: JSON – Produktbeschreibung Online-Shop	11

Dieses Dokument wurde mit Unterstützung von KI erstellt.

1 Einführung

Sowohl XML (Extensible Markup Language) als auch JSON (JavaScript Object Notation) sind weit verbreitete Formate zur Speicherung und zum Austausch strukturierter Daten. Sie dienen dazu, Informationen in einer für Menschen und Maschinen lesbaren Form darzustellen. Obwohl beide ähnliche Ziele verfolgen, unterscheiden sie sich in ihrer Struktur, Syntax und ihren Anwendungsbereichen. Dieses Handout bietet einen detaillierten Überblick über beide Formate und hebt ihre jeweiligen Stärken und Schwächen hervor.

2 XML (Extensible Markup Language)

2.1 Definition und Zweck

XML ist eine Auszeichnungssprache, die entwickelt wurde, um Daten zu transportieren und zu speichern. Der Fokus liegt dabei auf der Beschreibung der Datenstruktur und des Inhalts, nicht auf deren Darstellung. XML ist "extensible", was bedeutet, dass Benutzer ihre eigenen Tags definieren können, um die Daten entsprechend ihren spezifischen Anforderungen zu strukturieren.

2.2 Struktur und Syntax

XML-Dokumente bestehen aus einer Baumstruktur von Elementen. Jedes Element wird durch Start- und End-Tags begrenzt.

- **Elemente:** Der grundlegende Baustein eines XML-Dokuments. Sie können Attribute und andere Elemente (Kindelemente) enthalten.
- **Start-Tag:** <elementname>
- **End-Tag:** </elementname>
- **Leeres Element:** <elementname /> oder <elementname></elementname>
- **Attribute:** Bieten zusätzliche Informationen zu einem Element und werden innerhalb des Start-Tags in der Form attributname="wert" angegeben.
- **Wurzelement:** Jedes XML-Dokument muss genau ein Wurzelement haben, das alle anderen Elemente enthält.
- **Kommentare:** Werden mit <!-- ... --> eingefügt.
- **XML-Deklaration (optional):** Beginnt oft mit <?xml version="1.0" encoding="UTF-8"?> und gibt die XML-Version und die Zeichenkodierung an.

Beispiel:

```
<?xml version="1.0" encoding="UTF-8"?>
<person>
  <name vorname="Max" nachname="Mustermann" />
  <alter>30</alter>
  <adresse>
    <strasse>Musterstraße 1</strasse>
    <ort>Musterstadt</ort>
    <postleitzahl>12345</postleitzahl>
  </adresse>
  <hobbys>
    <hobby>Lesen</hobby>
    <hobby>Wandern</hobby>
  </hobbys>
</person>
```

2.3 Datentypen

XML selbst definiert keine spezifischen Datentypen. Die Bedeutung der Daten wird durch die Struktur und die Namen der Elemente und Attribute impliziert. Schemas (z.B. XML Schema Definition - XSD) können verwendet werden, um die Struktur und die erlaubten Datentypen eines XML-Dokuments zu definieren und zu validieren.

2.4 Validierung

Mit der Validierung wird geprüft, ob ein XML-Dokument einer bestimmten Struktur und Regelwerk entspricht. Dies erfolgt zusätzlich zur Überprüfung, ob das Dokument wohlgeformt ist.

2.4.1 Wohlgeformt

Ein XML-Dokument ist wohlgeformt, wenn es die grundlegenden Syntaxregeln von XML erfüllt. Dazu gehören:

- Es gibt genau ein Wurzelement.
- Jedes Start-Tag hat ein entsprechendes End-Tag.
- Die Elemente sind korrekt verschachtelt.
- Attribute sind korrekt formatiert (z.B. name="value").
- Sonderzeichen sind korrekt verwendet (z.B. & als &).

2.4.2 Gültig

Ein XML-Dokument ist gültig, wenn es nicht nur wohlgeformt ist, sondern auch den Regeln eines Schemas oder einer DTD (Document Type Definition) entspricht. Diese Regeln definieren die erlaubte Struktur, die Elemente und Attribute eines XML-Dokuments.

2.4.3 DTD (Document Type Definition)

DTD ist eine ältere Methode zur Definition der Struktur eines XML-Dokuments. Es beschreibt die erlaubten Elemente, deren Reihenfolge und Verschachtelung, sowie die erlaubten Attribute. DTDs sind weniger flexibel und mächtig als XML Schema (XSD), da sie keine Datentypen unterstützen, dafür sind sie kompakter.

Beispiel einer DTD:

```
<!ELEMENT person (name, alter, adresse, hobbies)>
<!ELEMENT name EMPTY>
<!ATTLIST name
  vorname CDATA #REQUIRED
  nachname CDATA #REQUIRED>
<!ELEMENT alter (#PCDATA)>
<!ELEMENT adresse (strasse, ort, postleitzahl)>
<!ELEMENT strasse (#PCDATA)>
<!ELEMENT ort (#PCDATA)>
<!ELEMENT postleitzahl (#PCDATA)>
<!ELEMENT hobbies (hobby+)>
<!ELEMENT hobby (#PCDATA)>
```

2.4.4 XSD (XML Schema Definition)

XSD ist eine modernere und leistungsfähigere Methode zur Definition der Struktur eines XML-Dokuments. Im Gegensatz zu DTDs unterstützen XSDs Datentypen, Namespaces und komplexe Strukturen. Sie sind in XML-Syntax geschrieben und bieten mehr Flexibilität und Präzision.

Beispiel eines XSD:

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="person">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="name">
          <xs:complexType>
            <xs:attribute name="vorname" type="xs:string" use="required"/>
            <xs:attribute name="nachname" type="xs:string" use="required"/>
          </xs:complexType>
        </xs:element>
        <xs:element name="alter" type="xs:integer"/>
        <xs:element name="adresse">
          <xs:complexType>
            <xs:sequence>
              <xs:element name="strasse" type="xs:string"/>
              <xs:element name="ort" type="xs:string"/>
              <xs:element name="postleitzahl" type="xs:string"/>
            </xs:sequence>
          </xs:complexType>
        </xs:element>
        <xs:element name="hobbies">
          <xs:complexType>
```

```
<xs:sequence>
  <xs:element name="hobby" type="xs:string" maxOccurs="unbound"/>
</xs:sequence>
</xs:complexType>
</xs:element>
</xs:sequence>
</xs:complexType>
</xs:element>
</xs:schema>
```

2.5 Vorteile von XML

- **Selbstbeschreibend:** Die Verwendung von Tags macht die Datenstruktur und den Inhalt klar und verständlich.
- **Erweiterbar:** Benutzer können ihre eigenen Tags definieren, um spezifische Datenanforderungen zu erfüllen.
- **Plattformunabhängig:** XML ist ein offener Standard und kann auf jeder Plattform und von jeder Anwendung verarbeitet werden.
- **Weit verbreitet:** XML ist ein etabliertes Format mit einer breiten Unterstützung durch Tools und Technologien.
- **Unterstützung für Schemas:** XML-Schemas ermöglichen die Validierung der Datenstruktur und -integrität.
- **Unterstützung für Transformationen:** Technologien wie XSLT (Extensible Stylesheet Language Transformations) ermöglichen die Umwandlung von XML-Daten in andere Formate.

2.6 Nachteile von XML

- **Verbosier:** Die Verwendung von öffnenden und schließenden Tags führt zu einer relativ großen Dateigröße im Vergleich zu anderen Formaten.
- **Komplexität:** Die Syntax kann für einfache Datendarstellungen unnötig komplex sein.
- **Parsing-Aufwand:** Das Parsen von XML-Dokumenten kann rechenintensiver sein als bei einfacheren Formaten.

2.7 Anwendungsbereiche von XML

- **Datenaustausch zwischen verschiedenen Systemen:** XML wird häufig für die Interoperabilität zwischen unterschiedlichen Anwendungen und Plattformen eingesetzt.
- **Konfigurationsdateien:** Viele Anwendungen verwenden XML-Dateien zur Speicherung von Konfigurationseinstellungen.

- **Dokumentenformate:** Formate wie DOCX (Microsoft Word) und ODT (OpenOffice Writer) basieren intern auf XML.
- **Webservices (SOAP):** XML ist die Grundlage für das SOAP-Protokoll (Simple Object Access Protocol) für Webservices.
- **Datenbanken:** Einige Datenbanken bieten native Unterstützung für die Speicherung und Abfrage von XML-Daten.

3 JSON (JavaScript Object Notation)

3.1 Definition und Zweck

JSON ist ein leichtgewichtiges Datenformat, das auf einer Teilmenge der JavaScript-Syntax basiert. Es wird hauptsächlich für die Übertragung von Daten zwischen einem Server und einer Webanwendung verwendet, dient aber auch zur Speicherung von Daten. JSON ist einfach zu lesen und zu schreiben und wird von vielen Programmiersprachen leicht verarbeitet.

3.2 Struktur und Syntax

JSON-Daten sind in Schlüssel-Wert-Paaren organisiert, die in Objekten und Arrays enthalten sein können.

- **Objekte:** Eine ungeordnete Sammlung von Schlüssel-Wert-Paaren. Objekte werden von geschweiften Klammern {} umschlossen. Schlüssel sind Strings (in doppelten Anführungszeichen), und Werte können primitive Datentypen (String, Zahl, Boolean, null), andere Objekte oder Arrays sein.
- **Arrays:** Eine geordnete Liste von Werten. Arrays werden von eckigen Klammern [] umschlossen. Die Werte in einem Array können beliebigen gültigen JSON-Datentyp haben.
- **Schlüssel-Wert-Paare:** Ein Schlüssel (String in doppelten Anführungszeichen) gefolgt von einem Doppelpunkt und dem zugehörigen Wert.
- **Datentypen:**
 - **String:** Eine Zeichenkette, die in doppelten Anführungszeichen steht.
 - **Number:** Eine numerische Zahl (Integer oder Fließkommazahl).
 - **Boolean:** true oder false.
 - **null:** Der Nullwert.
 - **Object:** Ein JSON-Objekt (siehe Beispiel).
 - **Array:** Ein JSON-Array (siehe Beispiel).

Beispiel:

```
{
  "name": {
    "vorname": "Max",
    "nachname": "Mustermann"
  },
  "alter": 30,
  "adresse": {
    "strasse": "Musterstraße 1",
    "ort": "Musterstadt",
    "postleitzahl": "12345"
  },
  "hobbys": [
    "Lesen",
    "Wandern"
  ]
}
```

3.3 Datentypen

JSON unterstützt die folgenden primitiven Datentypen: String, Number, Boolean und null. Darüber hinaus können komplexe Datenstrukturen durch die Verwendung von Objekten und Arrays gebildet werden.

3.4 Vorteile von JSON

- **Leichtgewichtig:** Die minimale Syntax führt zu kleineren Dateigrößen und schnellerer Übertragung.
- **Einfach zu lesen und zu schreiben:** Die Syntax ist für Menschen leicht verständlich.
- **Einfach zu parsen:** JSON wird von vielen Programmiersprachen nativ unterstützt, was das Parsen und die Verarbeitung sehr einfach macht.
- **Weit verbreitet im Web:** JSON ist das Standarddatenformat für die Datenübertragung in modernen Webanwendungen (insbesondere bei der Verwendung von RESTful APIs).
- **Direkte Abbildung auf Datenstrukturen:** JSON-Objekte und -Arrays lassen sich direkt auf die Datenstrukturen in vielen Programmiersprachen abbilden (z.B. Objekte und Listen/Dictionaries in Python, Objekte und Arrays in JavaScript).

3.5 Nachteile von JSON

- **Weniger selbstbeschreibend:** Ohne zusätzliche Metadaten (wie Schemas) kann die Bedeutung der Daten weniger offensichtlich sein als bei XML.
- **Keine Unterstützung für Attribute:** Zusätzliche Informationen zu Daten müssen als weitere Schlüssel-Wert-Paare innerhalb des Objekts dargestellt werden.
- **Keine Kommentare:** JSON unterstützt keine Kommentare in der Standarddefinition.

- **Keine Unterstützung für Schemas (standardmäßig):** Obwohl es JSON-Schema gibt, ist es nicht so weit verbreitet und standardisiert wie XML-Schema.

3.6 Anwendungsbereiche von JSON

- **Datenaustausch in Webanwendungen:** JSON ist das primäre Format für die Kommunikation zwischen Webbrowsern und Servern (z.B. bei AJAX-Anfragen).
- **APIs (RESTful):** Die meisten modernen RESTful APIs verwenden JSON für die Anfrage- und Antwortdaten.
- **Konfigurationsdateien:** Viele moderne Anwendungen verwenden JSON-Dateien für Konfigurationseinstellungen (z.B. package.json in Node.js-Projekten).
- **NoSQL-Datenbanken:** Einige NoSQL-Datenbanken (z.B. MongoDB) speichern Daten im JSON-ähnlichen BSON-Format.
- **Mobile Anwendungen:** JSON wird häufig für die Datenkommunikation in mobilen Apps verwendet.

4 Vergleichstabelle: XML vs. JSON

Merkmal	XML	JSON
Struktur	Baumstruktur mit Elementen und Attributen	Schlüssel-Wert-Paare in Objekten und Arrays
Syntax	Verbosier, mit Start- und End-Tags	Kompakt basierend auf JavaScript-Syntax
Lesbarkeit	Gut für Menschen lesbar	Sehr gut für Menschen lesbar
Selbstbeschreibung	Sehr gut durch Tags und Attributnamen	Weniger, benötigt oft zusätzliche Dokumentation
Größe	Relativ groß	Relativ klein
Parsing	Kann aufwendiger sein	Sehr einfach und schnell
Datentypen	Keine nativen, durch Schema definierbar	Definierte primitive Typen und Strukturen
Attribute	Unterstützt Attribute	Keine direkte Unterstützung für Attribute
Kommentare	Unterstützt Kommentare	Keine standardmäßige Unterstützung für Kommentare
Schemas	Weit verbreitete Unterstützung (XSD)	JSON-Schema existiert, weniger verbreitet
Transformation	Technologien wie XSLT verfügbar	Erfordert oft programmatische Lösungen
Web-Fokus	Weniger stark im modernen Web-Umfeld	Das Standardformat für Web-APIs
Anwendungsbereiche	Datenaustausch, Dokumentenformate, Konfigurationen	Webanwendungen, APIs, Konfigurationen, NoSQL

Sowohl XML als auch JSON sind wertvolle Formate für die Datenverarbeitung und den Datenaustausch. Die Wahl zwischen ihnen hängt oft von den spezifischen Anforderungen des jeweiligen Anwendungsfalls ab.

XML ist ideal, wenn eine hohe Selbstbeschreibung, die Validierung der Datenstruktur durch Schemas und die Möglichkeit zur Transformation der Daten wichtig sind. Es ist gut geeignet für komplexe Datenstrukturen und Szenarien, in denen Interoperabilität über verschiedene Plattformen hinweg entscheidend ist.

JSON ist die bessere Wahl, wenn es auf eine einfache, leichtgewichtige und schnell zu verarbeitende Datenübertragung ankommt, insbesondere im Kontext von Webanwendungen und APIs. Seine einfache Syntax und die direkte Abbildung auf Datenstrukturen in vielen Programmiersprachen machen es zu einer beliebten Wahl für moderne Anwendungen.

In vielen Fällen werden beide Formate in unterschiedlichen Kontexten verwendet, und das Verständnis ihrer jeweiligen Stärken und Schwächen ist entscheidend für die Auswahl des geeigneten Formats für eine bestimmte Aufgabe.

5 Komplexere Beispiele

In den zwei mitgelieferten Dateien finden Sie detailliertere Beispiele für XML und JSON.

5.1 Beispiel: XML – Stadtbibliothek

Erläuterungen zum XML-Beispiel:

- Das Wurzelement ist <bibliothek>.
- Es gibt einen <metadaten>-Bereich mit Informationen zur Bibliothek.
- Der <buecher>-Bereich enthält mehrere <buch>-Elemente, die jeweils ein einzelnes Buch repräsentieren.
- Jedes <buch>-Element hat ein id-Attribut und ein kategorie-Attribut.
- Innerhalb jedes <buch>-Elements gibt es verschachtelte Elemente wie <titel> (mit einem sprache-Attribut), <autor> (mit <vorname> und <nachname>), <verlag>, <jahr>, <seiten>, <preis> (mit einem waehrung-Attribut), <verfuegbarkeit> (mit Attributen für die Gesamt- und verliehene Anzahl), <rezensionen> (mit mehreren <rezension>-Elementen) und <auszeichnungen> oder <schlagwoerter> (abhängig vom Buch).
- Dieses Beispiel zeigt die Flexibilität von XML, komplexe, hierarchische Datenstrukturen mit Attributen zur weiteren Spezifizierung zu modellieren.

5.2 Beispiel: JSON – Produktbeschreibung Online-Shop

Erläuterungen zum JSON-Beispiel:

- Das Hauptobjekt enthält verschiedene Schlüssel-Wert-Paare, die die Eigenschaften des Produkts beschreiben.
- Es gibt verschachtelte Objekte wie preis, bewertungen, eigenschaften, verfuegbarkeit und die Elemente in den bilder-Array.
- bewertungen ist ein Array von Objekten, wobei jedes Objekt eine einzelne Bewertung darstellt.
- eigenschaften enthält weitere Details zum Produkt in einem verschachtelten Objekt.
- lieferumfang und bilder sind Arrays von Strings bzw. Objekten.
- Es werden verschiedene Datentypen verwendet: Strings, Zahlen (auch innerhalb von Objekten wie preis), Booleans (istImAngebot), Arrays und verschachtelte Objekte.

- Dieses Beispiel zeigt, wie JSON verwendet werden kann, um komplexe und detailierte Datenstrukturen für verschiedene Anwendungen darzustellen.